北京郵電大學

Queen Mary University of London

For examiners' use only

1	
2	
3	
4	
Total	

EBU6304 A

Joint Programme Examinations 2020/21

EBU6304 Software Engineering

Paper A

Time allowed 2 hours

Answer ALL questions

Complete the information below about yourself very carefully.

QM student number	
BUPT student number	
Class number	

NOT allowed: electronic calculators and electronic dictionaries.

INSTRUCTIONS

1. **You must NOT take answer books, used or unused, from the examination room.**
2. Write only with a black or blue pen **and in English**.
3. Do all rough work in the answer book – **do not tear out any pages**.
4. If you use Supplementary Answer Books, tie them to the end of this book.
5. Write clearly and legibly.
6. **Read the instructions on the inside cover.**

Examiners

Dr Ling Ma, Dr Luca Rossi, Dr Gokop Goteng, Dr Matthew Huntbach

Copyright © Beijing University of Posts and Telecommunications & © Queen Mary University of London 2021

Filename: 2021_EBU6304_A No answer book required

Software Engineering EBU6304

Previous test on QMPLUS 2021A

Solutions & Analysis

By Puyuan Zheng, BUPTIS.

(写在前面的话：解答题给分点均根据解析者本人对题目的理解认为的采分点。与实际若有不同，请以实际判分为准，本解析写出的采分点仅供参考。本解析若有问题错误，欢迎批评指正。)

Question 1: (25 Marks)

a) One of the problems with self-organising teams is that more experienced team members tend to dominate discussions and therefore to influence the team's way of working. Suggest two ways to solve this problem. (6 Marks)

【Answer】

First way, we should have small, co-located, multi-disciplinary team, and need members working around a table to have easy communication.

Second way, Pair programming. Programmers work in pairs, sitting together to develop code, which helps develop common ownership of code and spread knowledge across the team.

【考点】Agile Team

【解析】

读题，可知自己组织的团队一起合作，存在更有经验的统治影响团队的问题。需要我们提出两个方法解决这个问题。什么样的团队是好的？Agile team。但不能完全按 Agile team 的抄，结合问题根据问题找解决方案。

第一个角度，一般情况下有经验的人通常是在某一方面特别突出，这个时候为了防止他独导，我们需要多元化，让每一个人都能有发生的空间，坐在一起交流。

First way, we should have small, co-located, **multi-disciplinary team**, and need members **working around a table** to have **easy communication**.

核心：多职位，坐在一起，易于交流。

第二个角度，对于代码而言，我们要防止被一个人统治，我们需要统一制定规则，大家遵守这个规则。那么为了可以共同维持，我们采用 pair programming 的方式有助于做到这一点。

Second way, Pair programming. Programmers work in pairs, sitting together to develop code, which helps develop common ownership of code and spread knowledge across the team.

核心：小组编程，一起研发，一起建立共同的代码风格并传播到团队。

（6 Marks, 3 for each way.）

Agile team



- Small, co-located, multi-disciplinary team, members are usually working around a table
  - Easy communication
- Collective code ownership
- Common vision of system ('metaphor')
- Sustainable pace and common coding standard

Pair programming



- Programmers work in pairs, sitting together to develop code:
  - This helps develop common ownership of code and spreads knowledge across the team.
  - It serves as an informal review process, as each line of code is looked at by more than 1 person.
  - It encourages refactoring, as the whole team can benefit from this.
  - Measurements suggest that development productivity with pair programming is similar to (or more efficient than) that of two people working independently.

【*reference*】*ppt03 p11 p21*

b) Explain why estimating a user story using actual “hours” or “days” can lead to significant variations between the estimated effort and the actual effort. Suggest an estimating method to solve this problem with explanation. (6 Marks)

【Answer】

Because team members have different ability and experience.

Senior members may underestimate the time needed by more junior members to do the work.

Requirement may change due to business change, which leads to the change of estimating complete time.

Using story points to estimate the user stories.

Story points is an arbitrary measure that allows the team to understand the size of effort.

Fibonacci sequence: 1,2,3,5,8,13,21,...to represent the story points. As the points get higher, the degree of uncertainty is increasing.

Everyone on the team agrees to the size of story point, using relative sizing.

【考点】Estimating User Stories

【解析】

本题需要回答两个问题，为什么使用小时和天去估计 user story 的完成时间会和实际有很大出入？之后是提出一种方案去解决避免这个问题。

如果用绝对时间衡量，

首先的问题就是每个人能力不同完成同一个任务的时间也就不同。

Because team members have different ability and experience.

其次，一般制定计划的都是有经验的，他们可能会倾向凭借自己的经验高估手下人的水平，进而与实际造成较大出入。

Senior members may underestimate the time needed by more junior members to do the work.

此外，还有考虑需求在不断变化，user story 也会随时变化。

Requirement may change due to business change, which leads to the change of estimating complete time.

(3 Marks, 1 for each reason)

接下来第二个问题，推荐一个方式：

回顾Estimating的方法，Level of effort，ideal time，hours，story points.

Level of effort无法准确估计，只能有个大中小；ideal time会被低估；hours说了有问题；那么考虑story points.

那么就推荐Story points的方法，然后稍微展开解释即可。

Using story points to estimate the user stories. (4 Marks)

Story points is an arbitrary measure that allows the team to understand the size of effort.

Fibonacci sequence: 1,2,3,5,8,13,21,...to represent the story points. As the points get higher, the degree of uncertainty is increasing. (5 Marks)

Everyone on the team agrees to the size of story point, using relative sizing. (6 Marks)

【总结】Estimating user story的四种方法

1、Level of Effort or T-shirt sizing: Large, Medium, Small, Extreme Large, Extreme Small

2、Ideal time: under ideal circumstances

3、Hours: actual clock hours

4、Story point: An arbitrary measure that allows the teams to understand the size of effort.

最好的通用的：采用最后一种。

Estimating

- Estimates: how long the work is likely to take
- Some methods include:
  - Level of Effort or T-shirt sizing: small, medium, large, extra large…
  - Ideal Time: under ideal circumstance
  - Hours: actual clock hours
  - Story points: an arbitrary measure that allows the teams to understand the size of the effort

Estimating story points

- Story points:
  - Fibonacci Sequence: 1,2,3,5,8,13,21…
  - As the points get higher, the degree of uncertainty is increasing.
  - Abstract and negotiable
  - Everyone on the team agrees to the size of a story point, using relative sizing
    - E.g. adding a field to the database, agreed as a 5 story points



【reference】ppt04 p29 p30

c) What is low-fidelity prototyping? What are the benefits of using low-fidelity prototyping? (7 Marks)

【Answer】

Low-fidelity prototyping using paper and sketch to directly(intuitively) describe the interface of the software.

Benefits of using low-fidelity prototyping: Quick and Effective.

For Quick, No technical learning curve, Relevant feedback on early concepts and provides conceptual direction.

For Effective, Low investments of time/resources, Fail early, fail fast, Expedites detailed design.

【考点】Low-fidelity prototyping 相关内容

【解析】

说起 Low-fidelity prototype，我们想到的就是用纸和笔呈现。

问题要求回答什么是 low-fidelity prototype 以及这种方式的好处。好处 quick 和 effective（每个各 3 点详细展开即可）。

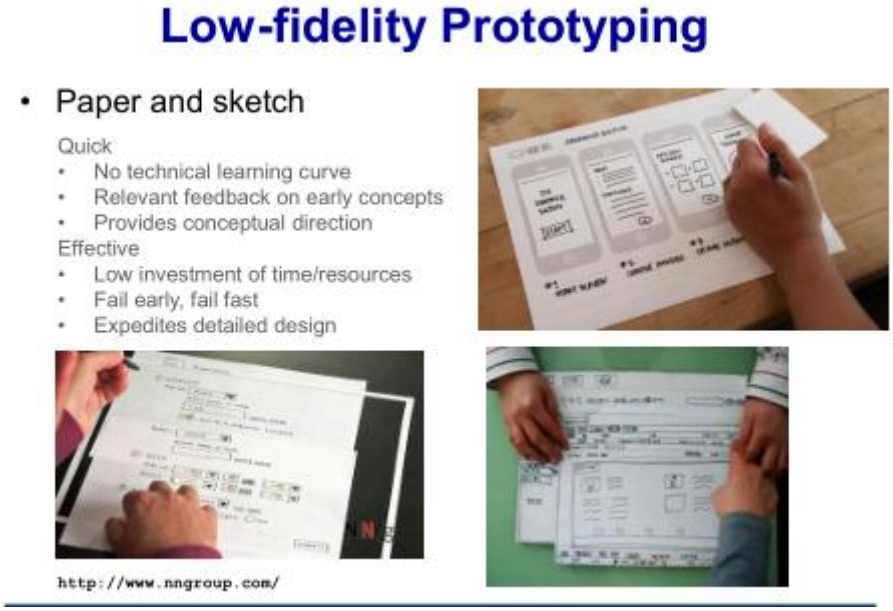
Low-fidelity prototyping using paper and sketch to directly(intuitively) describe the interface of the software. 核心点：纸笔，直观展现。

Benefits of using low-fidelity prototyping: Quick and Effective.

For Quick, No technical learning curve, Relevant feedback on early concepts and provides conceptual direction.

For Effective, Low investments of time/resources, Fail early, fail fast, Expedites detailed design.

中文理解：不用技术，反馈早，提供概念上的方向；花费成本少，试错成本低，加快细节设计。（6 Marks, 1 for each point, 3+3).



【拓展】Medium-fidelity, High-fidelity prototyping

Medium-fidelity prototyping:

Limited functionality but clickable areas which presents the interactions and navigation of an application. Usually built upon storyboards or user scenarios.

核心点：可以点击实现交互，功能有限，必须用一些 tool 了。

High-fidelity prototyping:

Computer-based interactive representation of the product in its closest resemblance to the final design in terms of details and functionality.

核心点：必须基于电脑制作，交互、细节和功能可以做到和最终设计非常接近、相似。

Medium-fidelity Prototyping

- Limited functionality but clickable areas which presents the interactions and navigation of an application.
- Usually built upon storyboards or user scenarios.
- Tools:
  - Fliplet.com
  - Proto.io
  - Adobe XD
  - And many more

High-fidelity Prototyping

- Computer-based interactive representation of the product in its closest resemblance to the final design in terms of details and functionality.
- Consider user interface (UI) and the user experience (UX).



【Reference】ppt05 p32-35

d) What is unit testing? Explain why it is essential to have a refactoring stage in the test driven development process in unit testing. (6 Marks)

【Answer】

Unit testing, also called component testing, is to test system components when we develop a class, method, etc.

In TDD process, we need refactoring every time after making changes to the code, in order to eliminate duplicate concepts and ensuring that the code is expressive.

【考点】Testing && TDD

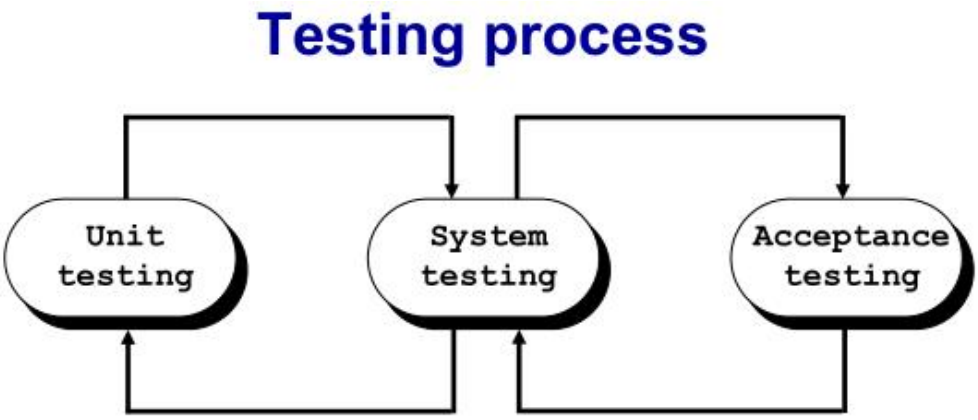
【解析】

本题考查 Testing 相关内容,首先问 Unit testing 是什么东西,以及 TDD 中 unit testing 过程中为什么 refactoring 很关键。

拆解问题：什么是 unit testing+tdd 中 unit testing 过程中为什么有 refactoring 很关键，2 问。

首先回答什么是 unit testing。

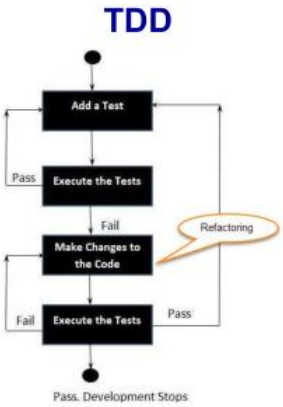
Unit testing, also called component testing, is to test system components when we develop a class, method, etc. (2 Marks)



- Unit testing (*aka* Component testing) → System components.
- System testing → The whole system.
- Acceptance testing (*aka* Alpha testing) → Customer's data.

接着回答在 TDD 中，unit testing 过程中 refactoring 为什么关键。

In TDD process, we need refactoring every time after making changes to the code, in order to **eliminate duplicate concepts** and **ensuring that the code is expressive**. (4 Marks, 2 for each).



Small cycle

- Write a small test to assert some piece of functionality.
- Demonstrate that the test fails.
- Write a small bit of code to make this test pass.
- Refactor both the test and code, eliminating duplicate concepts and ensuring that the code is expressive.

【reference】ppt09 p9; ppt10 p5 p9

Question 2: (25 Marks)

a) Answer the questions below about Analysis and Design: (13 Marks)

i) Briefly explain what entity and boundary classes are in the context of analysis classes. Give one example for each class. (5 Marks)

【Answer】

In the context of analysis classes,

Entity class include long-lived and persistent attributes, logical data structure and information that the system is dependent on.

Boundary class include receiving (presenting) information and requests from (and to) users and external systems.

Example of entity class:

```
import javax.persisitence.*;
import java.io.serializable;

@Entity(name = “BANKACCOUNT”)
public class BankAccount implements serializable{
    private String accountid;
    private String PIN;
    private double balance;
}
```

Example of boundary class: Windows, communication interfaces, pointer interfaces, sensors, terminal, etc.

【考点】Analysis Class

【解析】

读题可知，我们需要回答两个问题。

一是解释在 analysis classes 中 entity 和 boundary class 有什么东西。

In the context of analysis classes,

Entity class include **long-lived and persistent** attributes, logical data structure and **information that the system is dependent on**.(1 Mark)

核心点：永久属性，系统依赖的信息。

Boundary class include **receiving (presenting)** information and **requests from (and to)** users and external systems. (2 Marks)

核心点：收信息，从用户和外部系统请求信息。

二是各举一个例子。

Example of entity class: （实体类最好的方式直接用代码表达）

import javax.persisitence.*; //非常重要不可或缺

import java.io.serializable; //implements serializable 需要

@Entity(name = “BANKACCOUNT”) **//Entity 标识符**

```
public class BankAccount implements serializable{
    private String accountid;
    private String PIN;
    private double balance;
} //后面写一个类带几个 attribute 就 ok
```

(4 Marks)

Example of boundary class:

由于 Boundary class 没有代码实现，举例可以不写代码。

Windows, communication interfaces, pointer interfaces, sensors, terminal, etc. (5 Marks)

Entity classes

- Entity classes
  - Used to model information that is long-lived and persistent
    - Logical data structure
  - Information that the system is dependent on.
  - Store the data and define operations on the data.
    - Search, update, load, save, etc.
  - Data is stored in a database and represented by entity objects in memory.
    - Row in a table <=> object

Entity class example

```
import javax.persistence.*;
import java.io.Serializable;
import java.util.Date;

@Entity(name = "CUSTOMER")          //Name of the entity
public class Customer implements Serializable{
    private long custId;
    private String firstName;
    private String lastName;
    private String street;
    private String appt;
    private String city;
    private String zipCode;
    private String custType;
    private Date updatedTime;
…
```

Boundary classes

- Boundary classes
  - Used to model the interaction.
  - Often involve receiving (presenting) information and requests from (and to) users and external systems.
    - Deal with input and output, or connections with rest of system
  - Normally represent abstractions of user/device interface: windows, forms, communication interfaces, printer interfaces, sensors, terminals, etc.

【reference】ppt06 p11-p13

ii) Define and discuss the concepts of cohesion and coupling, explaining what an optimal system should look like in terms of applying these concepts. (7 Marks)

【Answer】

Concepts of cohesion and coupling

Cohesion: The number of dependencies within a subsystem, which indicates strength of interconnections.

Coupling: The number of dependencies between subsystems, which is a measure of the level of functional integration within a module.

As a optimal system, we should have loose coupling, and high cohesion.

Subsystem with as loosely as coupled to minimize the impacts of errors and future changes.

Subsystem with high cohesion means that all parts of the components should contribute to its logical function. If it is necessary to change the system, then everything to do with the component is encapsulated in one place.

【考点】Cohesion, Coupling 相关内容

【解析】

拆解本题问题，我们需要定义 cohesion 和 coupling 的概念，然后应用这些概念解释一个好的系统长什么样。

那么回答应该是先概念，然后结合概念回答好的系统长什么样。

概念部分：

Cohesion: The number of dependencies within a subsystem. （关键点：在一个 subsystem 之间），which indicates strength of interconnections.

Coupling: The number of dependencies between subsystems. （关键点：subsystems 之间，类与类之间），which is a measure of the **level of functional integration** within a module.

记忆方式：都是 dependencies 的数量，cohesion 一个类，coupling 多个类之间。Cohesion 衡量关联性强度，coupling 是整合度。

(4 Marks, 2 for each)

接下来利用这两个概念回答，好的系统要长成什么样。

As a optimal system, we should have loose coupling, and high cohesion. (5 Marks)

Subsystem with as loosely as coupled to minimize the impacts of errors and future changes.

要点：Loosely couple 的目的是减小系统出错的影响以及后续修改的困难。（不要牵一发动全身）(6 Marks)

Subsystem with high cohesion means that all parts of the components should contribute to its logical function. If it is necessary to change the system, then everything to do with the component is encapsulated in one place.

要点：所有部分必须服务于逻辑功能；如果需要改变，所有都要封装在一起。(7 Marks)

Coupling

- The number of dependencies between subsystems.
- Indicates strengths of interconnections
  - Tight: relatively dependent. Modifications to one is likely to have impact on others.
  - Loose: relatively independent. Modifications to one will have little impact on others.
- Ideally, subsystems are as loosely coupled as reasonable …
  - to minimise the impact on errors or future change.

Cohesion

- The number of dependencies within a subsystem.
- A measure of the level of functional integration within a module.
  - High: objects are related to each other and perform similar tasks.
  - Low: unrelated objects.
- Ideally, a subsystem should have high cohesion.
  - All parts of the component should contribute to its logical function.
  - If it is necessary to change the system, then everything to do with the component is encapsulated in one place.

【Reference】ppt07 p13 p15

iii) Fill in the blank: "Preventing direct access to some of an object's components is an example of _____ design principle."(1 Mark)

【Answer】Encapsulation

【考点】Design

【解析】

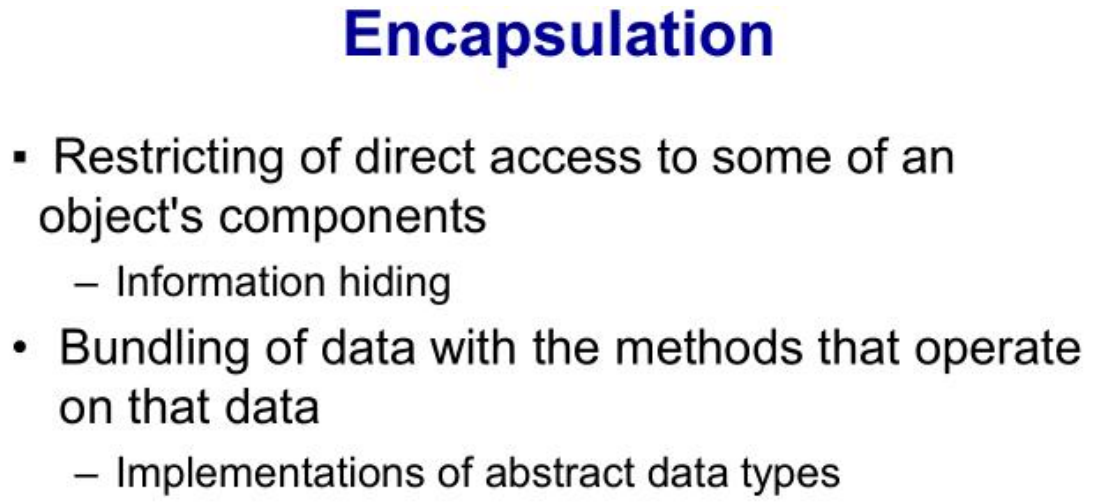
本题直白提问，防止直接访问一个对象的组成属于什么设计原则？根据关键词，我们不难看出 direct access，防止直接访问需要做什么？封装。

【误区！】很多人看到 design principle 会条件反射到 design principle 那六个上。但我们发现这六个原则说的核心没有一个对得上。那么考点此时判断就是错误的。结合题目的问题是关于 analysis 和 design 所以是在 tb2design 的部分去寻找相关概念。

【总结】封装的两个例子

1、Restricting of direct access to an object's components（Information Hiding）信息隐藏，限制访问其他对象

2、Bundling of data with the methods that operate on that data (Implementations of abstract data types). 信息捆绑，数据和使用数据的方法捆绑在一起。



【*Reference*】*ppt07 p11*

【拓展】六大设计原则 Six Design Principles

1、Single Responsibility Principle(SRP): Every object in a system should have a single responsibility, and all the object's service should be focused on carrying out that responsibility.

单一职责。一个系统每个对象都只有单一职责，所有对象都专注自己的职责。

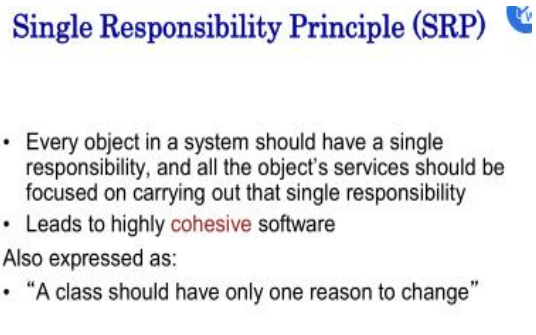
2、Open-Closed Principle(OCP): Software modules should be "Open for extension", "Close for modification".

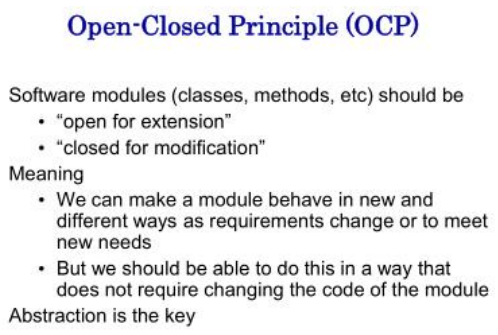
3、Don't Repeat Yourself(DRY)

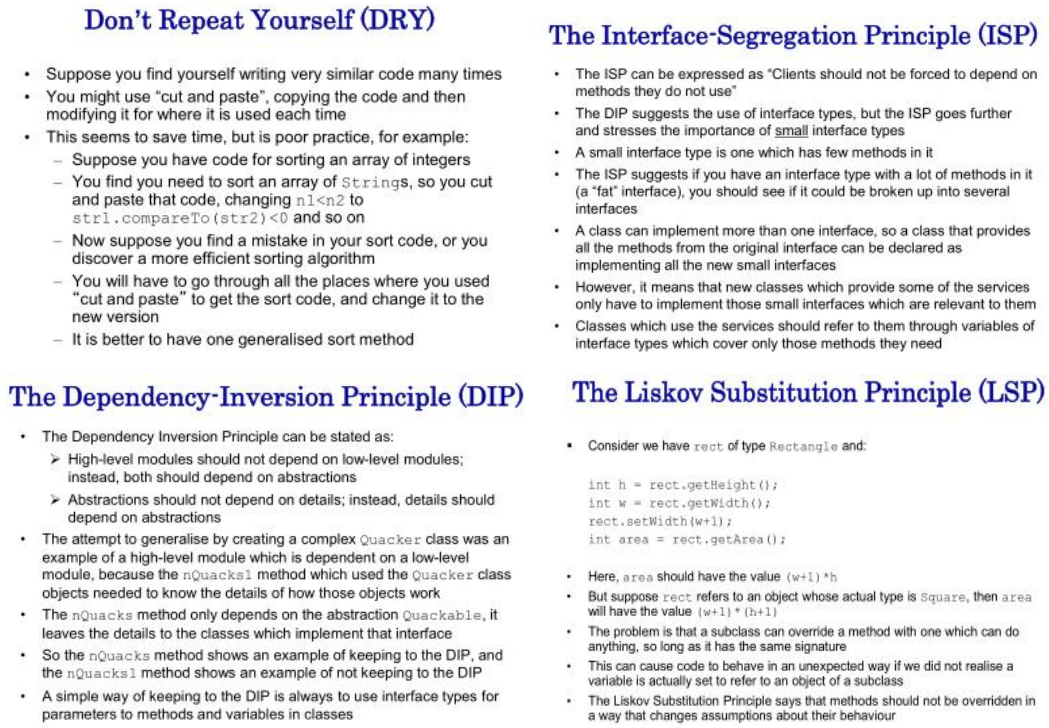
4、Dependency-Inversion Principle(DIP): High-level modules should not depend on low-level modules. Instead, both should depend on abstractions. Abstraction should not depend on details. Instead, details should depend on abstractions.

5、Interface-Segregation Principle(ISP): Clients should not be forced to depend on methods they do not use.

6、Liskov Substitution Principle(LSP): The methods should not be overridden in a way that changes assumptions about their behaviour. 方法不要重写改变他们假设的行为。







【Reference】ppt15 p43 p51 p57 p74 p76 p99

b) Answer the questions below about Software Architecture: (12 Marks)

i) What is meant by statelessness in the context of the REST architecture? (2 Marks)

【Answer】

There are no status in the communication between Client and Server. Stateless operation.

【考点】Software Architecture RESTful

【解析】

直白提问，REST 架构中无状态是什么意思？

核心要点：客户端和服务器之间交互的时候是无状态的。

There are no status in the communication between Client and Server. Stateless operation.

【注】课件中没有详细解释，本题参考某博客。上课老师提到一个 stateful 大概是就是客户端交互的时候会反馈一个状态。

【reference】https://blog.csdn.net/jmilk/article/details/50452595

ii) What is a principal design decision? What is an example of non-principal design decision? (2 Marks)

【Answer】

Principal design decision is the blueprint of a software system.

An example of non-principal design decision: Non-functional requirement.

【考点】Software Architecture

【解析】

本题提问直白，回答两个问题，principal design decision 是啥意思，以及举一个 non-principal design decision 的例子。

Principal design decision is the blueprint of a software system. (1 Mark)

不难知道，principal design decision，主要的设计决策在于构建软件架构。除此之外的东西，都是非软件架构的东西，这一部分均属于非主要设计决策。随便找个不涉及软件架构的例子写上即可。

An example of non-principal design decision: Non-functional requirement. (2 Marks)

- Software architecture refers to the set of principal design decisions, the blueprint of a software system



【reference】ppt12 p2

iii) Discuss the three main advantages of focusing on the software architecture. (6 Marks)

【Answer】

System Analysis: The analysis of whether the system can meet its non-functional requirements is possible.

Large-scale reuse: The architecture may be reusable across a range of systems (reuse is sometimes an economic necessity). Product-line architecture may be developed.

Stakeholder communication: Architecture may be used as a focus of discussion by system stakeholders.

(6 Marks, 2 marks for each)

【考点】Advantages of focusing on architecture

【解析】

本题直接考察聚焦软件结构的三个主要优点。直接分条作答，名称+描述即可。

1、System Analysis: The analysis of whether the system can meet its non-functional requirements.

记忆方法：系统分析。是否系统能够满足非功能性需求。

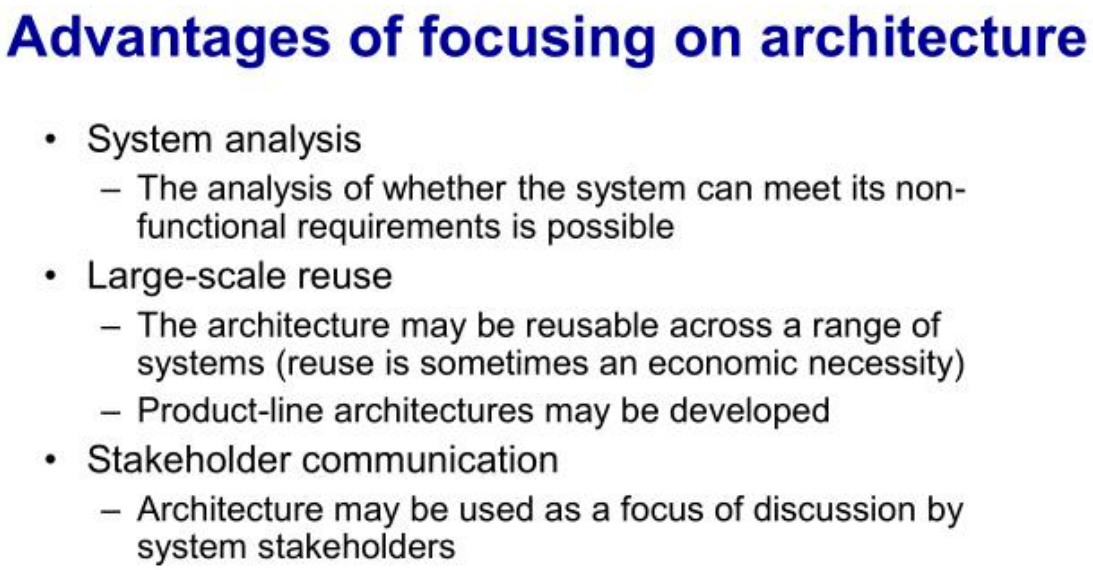
2、Large-scale reuse: The architecture may be reusable across a range of systems (reuse is sometimes an economic necessity). Product-line architecture may be developed.

记忆方法：大规模重复使用。大范围领域重复使用结构（是一种经济需要），流水线结构。

3、Stakeholder communication: Architecture may be used as a focus of discussion by system stakeholders.

记忆方法：参与者交流。系统参与者一起讨论出来集中的选项。

系统分析，大规模重复使用，参与者交流。



【reference】ppt12 p7

iv) What is the difference between a design pattern and an architectural pattern? (2 Marks)

【Answer】

The design pattern is general that we can reuse in many different circumstances to solve particular problems.

The architectural pattern is a stylized description of good design practice, which has been tried and tested in different environments.

【考点】Software Architecture

【解析】

本题直白提问，design pattern 和 architectural pattern 的不同。我们直接摆上概念，说明不同在哪里即可。

The design pattern is general that we can reuse in many different circumstances to solve particular problems.

The architectural pattern is a stylized description of good design practice, which has been tried and tested in different environments.

(2 Marks, 1 for each).

Architectural patterns (or styles)

- Patterns are a means of representing, sharing and reusing knowledge
- An architectural pattern is a stylized description of good design practice, which has been tried and tested in different environments
- Patterns should include information about when they are and when they are not useful
- Patterns may be represented using tabular and graphical descriptions

Design patterns

- These are just two examples of a “design pattern”, there are many more established design patterns
- Design patterns solve a particular problem, here:
  - How can we add an additional task to an existing method without changing the code of that method?
  - How can we fit a class into an interface which it is not declared as implementing without changing the code of that class?
- Design patterns are general, we can re-use the same pattern in many different circumstances
- Design patterns go beyond what is provided directly in the programming language
- Design patterns often involve a use of code which a novice programmer probably would not think of
- Design patterns provide us with a “vocabulary”, here we can talk to other programmers about using a “decorator” or an “adapter” and they will know what we mean

【*reference*】*ppt16 p12; ppt12 p11*

【**拓展**】design pattern, architectural pattern 以及 code pattern 三者的关系，上中下层级。



架构模式(Architectural Pattern)、设计模式(Design Pattern)、代码模式(Coding Pattern)

转载 cxzhq2002 于 2017-10-05 10:48:55 发布 4937 收藏 12

在做软件架构设计时,根据不同的抽象层次可分为三种不同层次的模式：架构模式(Architectural Pattern)、设计模式(Design Pattern)、代码模式(Coding Pattern)。

架构模式是一个系统的高层次策略，涉及到大尺度的组件以及整体性质和力学。架构模式的好坏可以影响到总体布局和框架性结构。

设计模式是中等尺度的结构策略。这些中等尺度的结构实现了一些大尺度组件的行为和它们之间的关系。模式的好坏不会影响到系统的总体布局和总体框架。设计模式定义出子系统或组件的微观结构。

代码模式（或成例）是特定的范例和与特定语言有关的编程技巧。代码模式的好坏会影响到一个中等尺度组件的内部、外部的结构或行为的底层细节，但不会影响到一个部件或子系统的中等尺度的结构，更不会影响到系统的总体布局和大尺度框架。

【reference】https://blog.csdn.net/cxzhq2002/article/details/78160530

Question 3: (25 Marks)

a) Answer the questions below about Project Management: (10 Marks)

i) Describe the distinctive characteristics of software products that make project management of software projects different from other engineering projects (for example building construction). (6 Marks)

【Answer】

【考点】Software Projects' Distinctions

【解析】

本题考查软件项目管理和其他工程项目管理独特的特点。直接描述特点即可拿分。软件项目管理因为什么特点会和其他的工程项目管理不同是需要我们作答的内容。

从软件项目的特点答起。

Software products are **intangible and flexible.** (2 Marks)

Software engineering is not **recognised as a sane engineering discipline**, which is **not standardised**. (3 Marks)

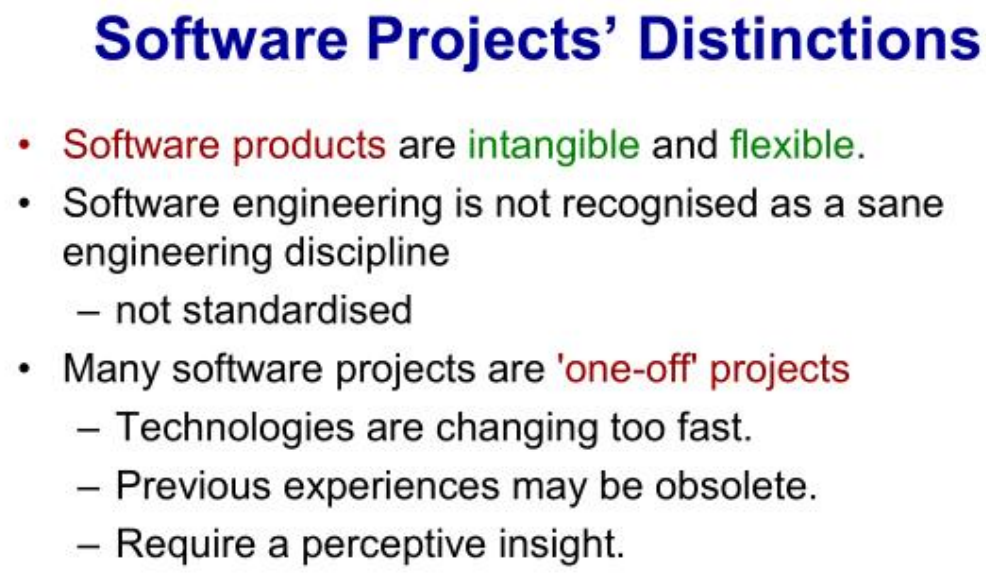
Many software projects are **'one-off' projects,** and need to face the problems of **technologies changing fast, previous experiences may be obsolete**. What's more, require a **perceptive insight.** (6 Marks)

核心点：无形的（工程项目有形），灵活的（工程项目比较固定）；

不像工程项目那样有标准化的健全的工程原则；

软件许多都是一次性项目，技术变化，过去的淘汰掉。（工程项目成熟的技术会一直运用，不太存在淘汰的可能）

同时还要感性的视角（工程项目需要非常理性）。



【reference】ppt13 p4

ii) In Agile project management, short daily stand-up meetings are important. Describe the purpose and activities of these daily stand-up meetings. (4 Marks)

【Answer】

Purpose: Adapted to incremental development and particular strength of agile methods.

Activities:

1) Tracks the backlog of work to be done

2) Records decisions

3) Measures progress against the backlog

4) Communicates with customers and management outside the team.

【考点】Agile project management, Scrum approach

【解析】

本题要求回答这些 meetings 的目的和内容。

Purpose: Adapted to **incremental development and particular strength** of agile methods. (2 Marks)

核心点：适应于 incremental development, particular strength.

Activities:

1) Tracks the backlog of work to be done

2) Records decisions

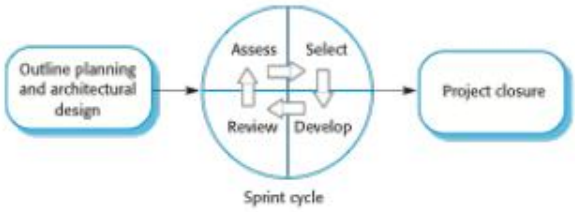
3) Measures progress against the backlog

4) Communicates with customers and management outside the team. (4 Marks)

记忆方式：追踪 backlog，记录决定，衡量进度，外界沟通。

Agile project management

- The standard approach to project management is plan-driven.
- Agile project management requires a different approach, which is adapted to incremental development and the particular strengths of agile methods. (Scrum approcah)



Teamwork

- The 'Scrum master' is a facilitator who
  - arranges short daily stand up meetings
  - tracks the backlog of work to be done
  - records decisions
  - measures progress against the backlog
  - communicates with customers and management outside of the team.
- Agile team
  - In most Agile teams, the golden number is said to be between 5 and 9 members

【reference】ppt13 p26 p27

b) Alpha & Beta is a small IT company that has been providing PC and desktop-based applications to its customers for the past 10 years. Due to the current COVID-19 pandemic, the company immediately moved all its applications to the Cloud as many customers work online remotely. Because of this sudden move to the Cloud, many customers are complaining that some of the software that used to work well on their PCs and desktops are not working properly and there is no alternative for a PC and desktop-based applications for them to use. In addition, the company does not have staff with the right skills to manage the Cloud-based applications as this is new to them. Answer the following questions. (10 Marks)

【考点】根据实际情境结合 Project Management 相关内容回答问题。

【解析】面对实际情境我们需要从中细致阅读，按顺序提取有效信息，并根据这些有效信息结合所学作答相关问题。

提取有效信息：

xx 公司提供 PC 和桌面端应用给客户多年了；（之前）

疫情影响之下，立即把所有的应用全部弄云端，因为更多客户远程线上工作了。（出现情况）

因为这个突然的转变，很多客户抱怨一些过去用得好好的东西全部不能用了，而且没有替代解决方案。（导致问题）

此外，公司没有拥有相关云应用技术的人，对于他们是个全新的事情。（无法解决）

i) Briefly discuss the problems that Alpha & Beta company is facing as a result of the sudden move to the Cloud-based applications due to COVID-19 pandemic (use professional software engineering terminologies). (3 Marks)

【Answer】

The problems facing:

1、Some of the software projects, called **"One-off projects"**, that used to work well on PCs and desktops are not working properly after moving to Cloud suddenly due to the fast change of technologies.

2、Due to **poor people management**, the company don't have staff with right skills to manage.

3、No alternative for PCs and desktops application for them to use, which are **obsolete.**

【解析】

本题要求简单描述这个公司突然转云端带来的问题，并使用专业的软工术语回答。

首先我们要知道什么问题，再用所学进行属于匹配。

直白话说：原来桌面端的东西不能用了，没有替代解决方案。在这个情况下，公司没有相关人士进行解决。

把这个东西结合术语解释即得答案。

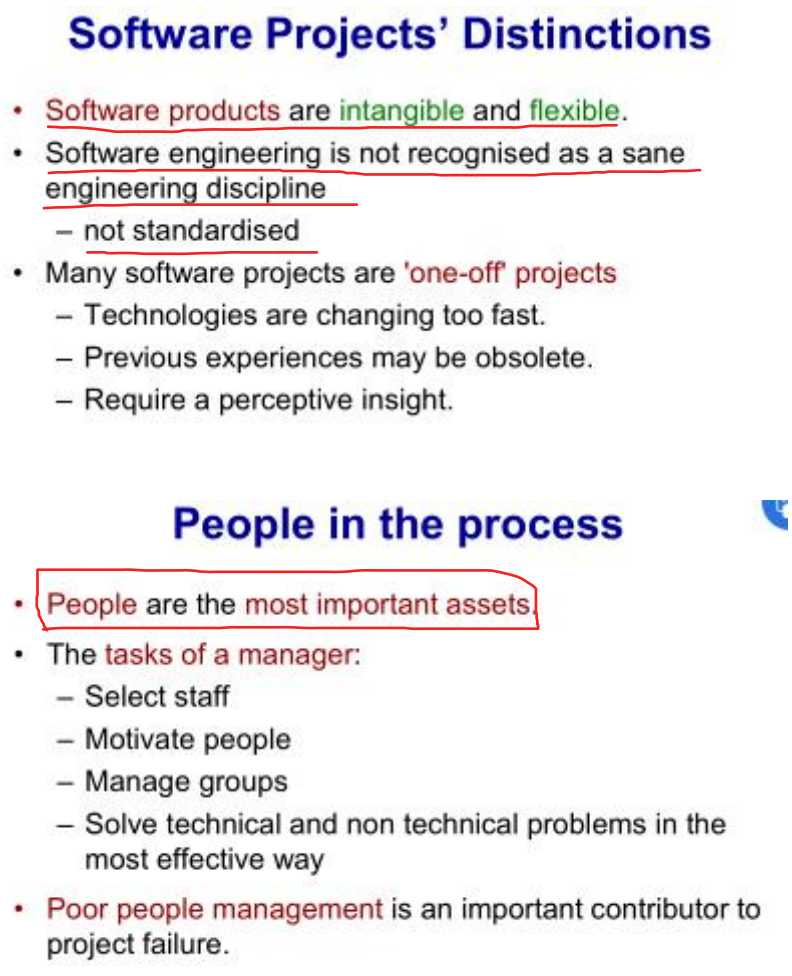
The problems facing:

1、Some of the software projects, called “**One-off projects**”, that used to work well on PCs and desktops are not working properly after moving to Cloud suddenly due to the fast change of technologies.

2、Due to **poor people management,** the company don’t have staff with right skills to manage.

3、No alternative for PCs and desktops application for them to use, which are **obsolete.**

(3 Marks, 1 for each)



【reference】ppt13 p4 p22

ii) Discuss what Alpha & Beta IT company failed to do which could have prevented the problem they are now facing. (3 Marks)

【Answer】

1、The company haven’t follow the change of business environment to do some work to adapt to the change and always remain the past pattern that will be replaced in the future.

2、After the sudden change, the alternative way to solve the problem haven’t studied in time.

3、The company haven’t recruit some staff who has the knowledge of cloud-based application during the sudden change. Present staff haven’t learn the relative knowledge in time.

【解析】

本题要求回答这个公司出现了问题，什么可以避免的地方没有做好。

很显然，经过刚才分析，我们可以得知：

过去这种软件服务了 10 年，说明一直保持这个模式，没有改变。在疫情这种突变下，公司没有与时俱进，跟随改变。

突然上云之后，没有及时出替代的方案供用户使用过去不能用的软件。

公司现在没有相关技术的人，没有去及时招聘具备相关资历的人才。

根据上述分析，我们可以得到如下答案：

4、The company haven’t follow the change of business environment to do some work to adapt to the change and always remain the past pattern that will be replaced in the future.

5、After the sudden change, the alternative way to solve the problem haven’t studied in time.

6、The company haven’t recruit some staff who has the knowledge of cloud-based application during the sudden change. Present staff haven’t learn the relative knowledge in time.

(3 Marks, 1 for each)

iii) Suggest two solutions to the company on how to improve their management. (4 Marks)

【Answer】

First Solution: For future planning, the company should add some schedule for the unanticipated problem (like the pandemic).

Second Solution: The company should select staff which has knowledge of cloud-based application and to seek for the alternative way for the present situation.

【解析】

本题希望提出两个方案，公司该如何改进管理。

要改进，肯定要针对问题对症下药。

总结来看：没出替代方案和没有相关人员。

从 Project Management 的任务中进行分析，我们需要进行 Scheduling 以及 Personnel Selection 的相关提高。

First Solution: For future planning, the company should add some schedule for the unanticipated problem (like the pandemic).

Second Solution: The company should select staff which has knowledge of cloud-based application and to seek for the alternative way for the present situation.

(4 Marks, 2 for each)

Scheduling problems

- Do not assume that every stage will be problem free; the unexpected always happens, always allow for a contingency in planning
  - People, hardware, software
- A good rule
  - Estimate as if nothing will go wrong.
  - Add 30% for anticipated problems.
  - Add further 20% to cover unanticipated problems.

People in the process

- People are the most important assets.
- The tasks of a manager:
  - Select staff
  - Motivate people
  - Manage groups
  - Solve technical and non technical problems in the most effective way
- Poor people management is an important contributor to project failure.

【reference】ppt13 p13 p22

c) Describe the four freedoms of Free Software and the benefit of having Free Software. (5 Marks)

【Answer】

Four Freedoms of Free Software:

1) The freedom to **run the program**, for any purpose.

2) The freedom to **study how the program works**, and **change it** so it does your computing as you wish. Access to the source code is a precondition for this.

3) The freedom to **redistribute copies** so you can help your neighbour.

4) The freedom to **distribute copies** of your modified versions to others. By doing this you can give the whole community a chance to benefit from your changes. Access to the source code is a precondition for this.
Benefit of having Freedom Software: Enjoyment of software and/or personal freedom, and making or having powerful, reliable software.

【考点】Software Freedom

【解析】

本题我们需要回答两个问题：freedoms of Free Software 以及 Free Software 的好处。

Four Freedoms of Free Software:

5) The freedom to **run the program**, for any purpose.

6) The freedom to **study how the program works**, and **change it** so it does your computing as you wish. Access to the source code is a precondition for this.

7) The freedom to **redistribute copies** so you can help your neighbour.

8) The freedom to **distribute copies** of your modified versions to others. By doing this you can give the whole community a chance to benefit from your changes. Access to the source code is a precondition for this.
(4 Marks, 1 for each)

核心：运行，研究怎么运行，重分配，分配修改后的版本（整个社区受益）

Benefit of having Freedom Software: Enjoyment of software and/or personal freedom, and making or having powerful, reliable software. (5 Marks)

Software Freedom

- The strongest form of "free software" insists on the following four freedoms:
  - The freedom to run the program, for any purpose
  - The freedom to study how the program works, and change it so it does your computing as you wish. Access to the source code is a precondition for this.
  - The freedom to redistribute copies so you can help your neighbour.
  - The freedom to distribute copies of your modified versions to others. By doing this you can give the whole community a chance to benefit from your changes. Access to the source code is a precondition for this.
- These were developed by Richard Stallman founder of the GNU project, which was set up to produce an OSS version of the operating system Unix
- The work co-ordinated by Stallman was combined with the work on a Unix kernel initiated by Linus Torvald to produce the system which is generally called Linux, but Stallman prefers to call GNU/Linux
- Stallman is know for his strong personal views, and has established a distinction between Free Software meaning with all the freedoms above, and "Open Source Software" which may have more limited freedoms

【reference】ppt17 p9

Question 4: (25 Marks)

a) An important aspect of good quality software development is to reduce dependency between different parts of a program. Explain what is meant by this, and why it is important. Explain how the general idea of object-oriented programming fits in with this. (8 Marks)

【Answer】

Reduce dependency between different parts of a program means that enable details connected to implementation to be hidden from code which use objects, in line with Dependency Inversion Principle (DIP).

The importance of doing this is that an important part of good program design is to work out what abstractions we want to define and use, what are the important aspects of some type of object that we need to consider. Reduce dependency is working out the important aspects and remove the unimportant details.

With the Object Pool and Flyweight design patterns, the code which uses objects does not have to manage when to allow aliasing to save space.
With the State design pattern, the code which uses objects does not have to manage the transition between one representation and another when for efficiency reasons it is a good idea to change
With the Observer design pattern, the code which uses objects does not have to manage the details of communicating the effects of method calls on objects to code which displays or reports those effects.

【考点】Design Patterns Reduced Dependency

【解析】

拆解本题，我们可知需要回答如下问题：

1、解释 reduce dependency between different parts of a program 啥意思

2、为什么这么做很重要

3、OOP 的理念是怎么符合这一点的

第一个问题，解释。

Reduce dependency between different parts of a program means that enable details connected to implementation to be hidden from code which use objects, in line with Dependency Inversion Principle (DIP).(2 Marks)

核心点：hidden from code which uses objects, DIP 原则。

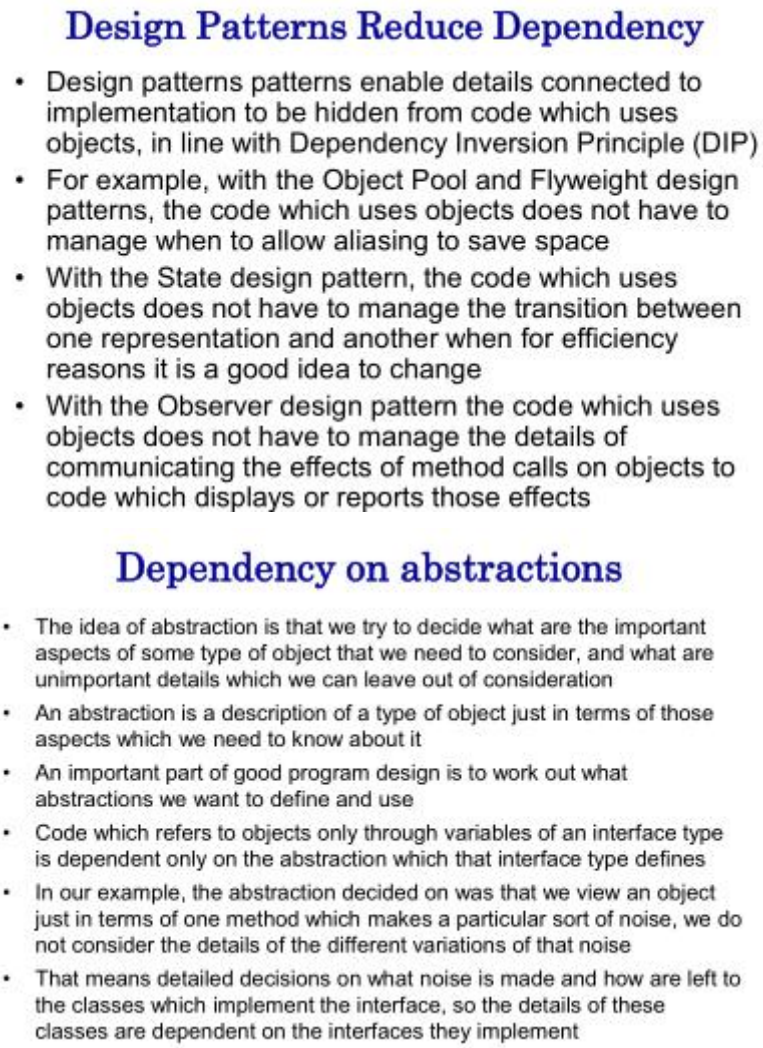
接着回答为什么这一点很重要。根据概念我们可以考虑从符合 DIP 原则的好处回答重要性。

The importance of doing this is that an important part of good program design is to work out what abstractions we want to define and use, what are the important aspects of some type of object that we need to consider. Reduce dependency is working out the important aspects and remove the unimportant details. (5 Marks)

最后回答 OOP 理念是如何符合这一点，那么我们要从 OOP 的实质出发。

Object-Oriented Programming，基于 object 的编程，那么使用 Object 编程为什么符合这一点。那么就是回答的方式，使用 object 怎么怎么符合这一点。

With the Object Pool and Flyweight design patterns, the code which uses objects does not have to manage when to allow aliasing to save space.
With the State design pattern, the code which uses objects does not have to manage the transition between one representation and another when for efficiency reasons it is a good idea to change
With the Observer design pattern, the code which uses objects does not have to manage the details of communicating the effects of method calls on objects to code which displays or reports those effects.
核心点：1、不需要考虑偏移节约空间的问题；2、不需要考虑转换表达的问题；3、不需要考虑调用细节的问题。
(8 Marks)



【reference】ppt16 p63; ppt15 p75

b) What is the general idea of a design pattern in software development? (4 Marks)

【Answer】

【考点】Design Pattern 的内涵

【解析】

本题直白提问，design pattern 在软件开发中的想法。不是纯含义从软件开发的角度选择点作答。

Design pattern is to solve a particular problem, like how can we add an additional task to an existing method without changing the code of that method and how can we fit a class into an interface which it is not declared as implementing without changing the code of that class.

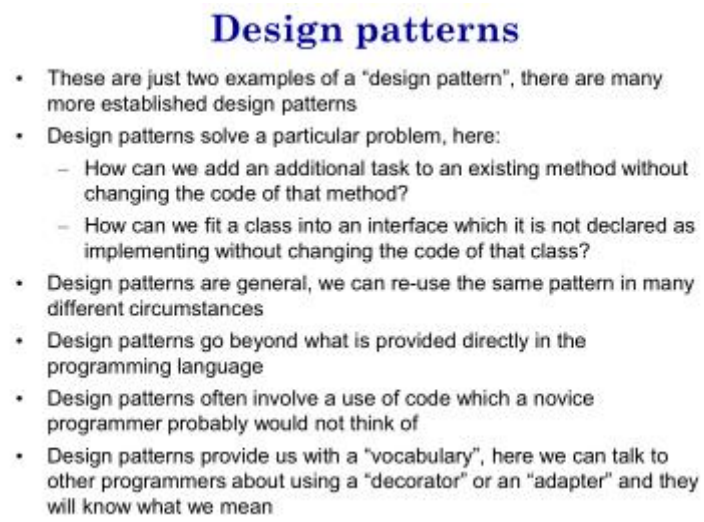
Design patterns go beyond what is provided directly in the programming language, and can be reuse the same pattern in different circumstances.

Design patterns often involve a use of code which a novice programmer probably would not think of.

Design patterns provide us with a "Vocabulary", here we can talk to other programmers about using a "decorator" or an "adapter" and they will know what we mean.

核心点：解决具体问题（怎么在现有方法不改代码情况添加任务，如何加入一个类）；可以多场景反复使用；包含新手想不到的代码使用方法；词汇表（如何使用 decorator 或者 adapter）。4 点。

(4 Marks, 1 for each)



【reference】ppt16 p12

c) Give a full explanation of the Decorator design pattern, and explain how it differs from the Adapter design pattern. (8 Marks)

【Answer】

The Decorator design pattern involves:

A class which implements an interface.

Inside it a variable of the same interface type whose value is set in the constructor.

Each method from the interface has code which calls the same method with the same arguments on that variable, plus some extra work.

The effect is to "decorate" the methods of an object of the interface type with the code that does the extra work.

Difference from the Adapter design pattern:

1) The Adapter design pattern "adapts" an object of one class to fit into an interface when the object is not of a type which implements the interface. The decorate design pattern involves a class which implements an interface.

2) It works by "wrapping" the object as an inner object in an outer object which is of a class that does implement the interface. Decorate design pattern works by "Decorating" for extra work.

3) Each method in the interface has to be implemented by code which performs what is regarded as the “equivalent” in the class of the inner object. Decorate design pattern works on the same method with the same arguments, and plus the extra work.

【考点】Design Pattern

【解析】

本题要求回答两个问题：给出 Decorator design pattern 的完整解释，并且解释和 Adapter design pattern 的不同。

完整解释：

The Decorator design pattern involves:

A class which implements an interface.

Inside it a variable of the same interface type whose value is set in the constructor.

Each method from the interface has code which calls the same method with the same arguments on that variable, plus some extra work.

The effect is to “decorate” the methods of an object of the interface type with the code that does the extra work.(4 Marks)

核心点：包含 3 个东西+效果。实现界面的类，在这之中的构造器设置了一个有同样的界面的值，界面里的每个方法参数用法都一样；decorate 的效果就是 do extra work 的方法。

接着说不同。

Difference from the Adapter design pattern:

4) The Adapter design pattern “adapts” an object of one class to fit into an interface when the object is not of a type which implements the interface. The decorate design pattern involves a class which implements an interface.

实体不实现界面与实现界面的区别。

5) It works by “wrapping” the object as an inner object in an outer object which is of a class that does implement the interface. Decorate design pattern works by “Decorating” for extra work.

Wrapping 和 Decorating 的区别。

6) Each method in the interface has to be implemented by code which performs what is regarded as the “equivalent” in the class of the inner object. Decorate design pattern works on the same method with the same arguments, and plus the extra work.

界面方法的实现：等价类和相同方法参数+额外工作的不同。(8 Marks)

The Decorator Design pattern

- This is an example of the Decorator design pattern
- The Decorator design pattern involves
  - A class which implements an interface
  - Inside it a variable of the same interface type whose value is set in the constructor
  - Each method from the interface has code which calls the same method with the same arguments on that variable, plus some extra work
- In this example, the extra work is to increase the value of the count by one
- The effect is to "decorate" the methods of an object of the interface type with the code that does the extra work

The Adapter design pattern

- The Adapter design pattern "adapts" an object of one class to fit into an interface when the object is not of a type which implements the interface
- It works by "wrapping" the object as an inner object in an outer object which is of a class that does implement the interface
- Each method in the interface has to be implemented by code which performs what is regarded as the "equivalent" in the class of the inner object
- Example: class `Goose` has a method

  ```
  public void honk() …
  ```

- which is judged equivalent to the `quack()` method of interface `Quackable`

激活 Windows

【*reference*】*ppt16 p5 p7*

d) In what way does using the Decorator and Adapter design patterns fit in with the general idea of the Open Closed Principle (OCP) of program design? (5 Marks)

【Answer】

【考点】Design Pattern, Design Principle 综合考察

【解析】

本题要求回答使用这两种 design pattern 如何契合 OCP 原则的思想。

首先我们要知道 OCP 原则什么思想——Open for extension, Close for modification. 不修改，去拓展。

Decorator Method 的核心在于做一些额外的工作。

Adapter Method 的核心在于适应到不同类型的实体，并且不改原代码写等价目的的代码。

结合上述分析，我们可以做出如下回答：

The general idea of OCP is Open for extension and Close for modification. (1 Mark)

For Decorator design pattern, the key is to **plus some extra work** on the methods that existed. To **decorate the methods of an object of the interface type with the code that does the extra work**. Which doesn't change the original code and extend something new, which is the extra work. (3 Marks)

For Adapter design pattern, the key is to adapts an object of one class to fit into an interface with the object that is not of type which implements the interface, which is working by **"Wrapping" the object as an inner object in an outer object** which is of a class that does implement the interface. Each method in the interface has to be implemented by code which performs what is regarded as the **"equivalent" in the class** of inner object. Which doesn't change the original code and extend something new as class with equivalent effect. (5 Marks)

Open-Closed Principle (OCP)

Software modules (classes, methods, etc) should be

- "open for extension"
- "closed for modification"

Meaning

- We can make a module behave in new and different ways as requirements change or to meet new needs
- But we should be able to do this in a way that does not require changing the code of the module

Abstraction is the key

The Decorator Design pattern

- This is an example of the Decorator design pattern
- The Decorator design pattern involves
  - A class which implements an interface
  - Inside it a variable of the same interface type whose value is set in the constructor
  - Each method from the interface has code which calls the same method with the same arguments on that variable, plus some extra work
- In this example, the extra work is to increase the value of the count by one
- The effect is to "decorate" the methods of an object of the interface type with the code that does the extra work

The Adapter design pattern

- The Adapter design pattern "adapts" an object of one class to fit into an interface when the object is not of a type which implements the interface
- It works by "wrapping" the object as an inner object in an outer object which is of a class that does implement the interface
- Each method in the interface has to be implemented by code which performs what is regarded as the "equivalent" in the class of the inner object
- Example: class `Goose` has a method

```
public void honk() …
```

- which is judged equivalent to the `quack()` method of interface `Quackable`

【reference】ppt15 p53; ppt16 p5 p7